CSCI-C 212 Practice Exam 2 Spring 2026 (80 points)
Mar 30, 2026

PLEASE READ ALL DIRECTIONS BEFORE STARTING YOUR EXAM. DO NOT OPEN UNTIL YOU ARE TOLD TO DO SO.

This is a closed-note exam except for your approved aid sheet. No electronic devices or communication with anyone other than proctors and the professor is allowed. *Cheating or talking with others will result in an automatic F in the course.*

Unless a question states otherwise, you may use any Java class without writing its corresponding `import`. You do not need to write the full design recipe (signature, documentation, tests), though you may do so if it helps.

If you believe you have found an error on the exam, raise your hand and notify a proctor.

When you are finished, check over your work. If more than 10 minutes remain, submit your exam and aid sheet to a proctor, show your ID, then quietly exit. No one may leave during the final 10 minutes.

Do not tear out, detach, or remove any pages from this exam. We will deduct 5 points (out of 80) for each page removed. If you use the scratch sheet of paper, please label the question on the sheet and put a note on the question.

Do not waste time erasing; simply cross out your work and we will ignore it.

Illegible hand-writing will receive a 0.

The exam contains 80 points.

You have **75 minutes** to complete the exam.

Good luck!

Question	Points	Score
1	20	
2	30	
3	30	
Total:	80	

Name: _____

IU Email: _____

1. (20 points) Design the `Set<String> uniqueShortestWords(List<String> words, int k)` method that, when given a list of strings and an integer $k \geq 1$, returns a set of size at most k of the strings that contain the fewest characters. If two strings contain the same number of characters, e.g., `"dog"` and `"cat"`, then prioritize the string that comes first lexicographically.

   Hint: Use a `PriorityQueue` with a custom `Comparator`!

```
/**
 * Returns at most k shortest words from the given list. If two words have the
 * same length then the one that comes lexicographically first is prioritzed.
 * @param words list of words.
 * @param k integer >= 1.
 * @return list of length at most k.
 */
Set<String> uniqueShortestWords(List<String> words, int k) {









}
```

2. (30 points) This question has two parts, each of which are weighted equally.

   Two integers in a list of integers are *valley-contributing* if the integer between them is strictly greater than both. For example, in the list $[3, 5, 18, 4, 1, 2]$, 5 and 4 are valley-contributing because the number in between, namely 18, is larger than both 5 and 4.

   (a) *(15 points)* Design the *tail recursive* `Optional<Integer> countValleyContributingTR-(List<Integer> ls)` method that counts the number of valley-contributing integers.

   Using the above list as an example, we would return an `Optional` containing `2`. If there are fewer than 3 values in the given list, return an empty `Optional`. You will need to design a helper method. Remember the relevant access modifiers!

   Hint: We don't require that your helper method return an `Optional`; only that your driver does. So, as long as your helper is tail recursive, you can process its return value however you'd like in the driver, e.g., with a case analysis.

   Your helper method must be tail recursive or you will receive no credit.

(b) *(15 points)* Design the *iterative* `Optional<Integer> countValleyContributingLoop-(List<Integer> ls)` method that solves the same problem as (a), but uses a loop.

Your method must use a loop and not be recursive or you will receive no credit.

Before proceeding...

For the last question, did you use the translation pipeline to solve the problem? It was not required; I am just curious!

_____ Yes

_____ No

3. (30 points) A *range* describes an interval between two types of ordered objects. For example, the range from 3 to 67 describes a range where the minimum is 3 and the maximum is 67. We trivially know how to compare integers, but we can extend the idea of a range to any type that can be compared. In this question, you will implement a generic class called `Range` to represent an interval.

   (a) *(5 points)* Fill in the `Range` class fields and constructor. The constructor receives three values: `T min`, `T max`, and `Comparator<T> cmp`. The first and second parameters designate the minimum and maximum values of the `Range`, and `cmp` denotes how to compare objects of type `T`. Store these three parameters as `final` instance variables. You may assume that `min` is always strictly less than `max` according to `cmp`.

```
class ___________________________________ {

  __________________ __________________ __________________ MIN;
  __________________ __________________ __________________ MAX;
  __________________ __________________ __________________ CMP;

  // Constructor.
  ______________________(_____________________________________________________) {
    ______________________ = ___________________________________________;
    _____________ = ______________________________;
    _____________ = ______________________________;
  }
}
```

(b) *(5 points)* Design the `boolean strictlyContains(T v)` method, which returns whether v is strictly contained in `this` range. A value v is strictly contained within the `Range` if `min` $< v <$ `max`.

Hint: `Comparator<T>` objects have the `compare` method that you can invoke on the object. Pass to it two values t_1 and t_2 of type `T`, and it returns a negative integer if $t_1 < t_2$, a positive integer if $t_1 > t_2$, and 0 if t_1 is equal to t_2.

(c) *(10 points)* Design the `boolean overlapsWith(Range<T> othRange)` method that returns whether `this` range overlaps with the given `Range`. Note that this should be a strict overlap and not simply "touching." For example, a range whose minimum is 3 and maximum is 67 does not overlap with a range whose minimum is 67 and whose maximum is 71.

(d) *(5 points)* Override the `equals` method, which returns whether this `Range` is equal to another object. Two `Range` instances are equal if and only if they have equal minimum and maximum values.

```
  @Override

__________________ equals(____________________) {
  if (_____________________________________________) {
    return ________________________;
  } else {
    ____________________________________________________;
    return ____________________________________________________________;
  }
}
```

(e) *(5 points)* Override the `hashCode` method, which returns the hash code of this `Range`. The hash code is determined by invoking `Objects.hash` on the minimum and maximum values.

```
@Override
______________________ ______________________ hashCode() {
  return ____________________________________________________________;
}
```

Scratch work